

Principles of Programming Languages Logic Programming.

Andrei Arusoaie¹

¹Department of Computer Science

January 11, 2021

Outline

Paradigms

Outline

Paradigms

Logic Programming

Paradigms

- ▶ Imperative Programming: ✓

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming: ✓

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming: ✓
- ▶ Functional Programming: ✓

Paradigms

- ▶ Imperative Programming: ✓
- ▶ Object Oriented Programming: ✓
- ▶ Functional Programming: ✓
- ▶ Logic Programming

General facts about PLs

General facts about PLs

We have seen PLs where the computations are based on:

General facts about PLs

We have seen PLs where the computations are based on:

- ▶ *state + modifiable variable + assignment*

General facts about PLs

We have seen PLs where the computations are based on:

- ▶ *state + modifiable variable + assignment*
- ▶ Lambda calculus + higher-order functions + recursion

General facts about PLs

We have seen PLs where the computations are based on:

- ▶ *state + modifiable variable + assignment*
- ▶ Lambda calculus + higher-order functions + recursion

Are there other approaches?

General facts about PLs

We have seen PLs where the computations are based on:

- ▶ *state + modifiable variable + assignment*
- ▶ Lambda calculus + higher-order functions + recursion

Are there other approaches?

- ▶ Yes

General facts about PLs

We have seen PLs where the computations are based on:

- ▶ *state + modifiable variable + assignment*
- ▶ Lambda calculus + higher-order functions + recursion

Are there other approaches?

- ▶ Yes: Logic programming

Computation vs. Deduction

Computation vs. Deduction

- ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)

Computation vs. Deduction

- ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)
- ▶ Deduction: requires more than automatic application of rules

Computation vs. Deduction

- ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)
- ▶ Deduction: requires more than automatic application of rules, i.e., needs human ingenuity to complete

Computation vs. Deduction

- ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)
- ▶ Deduction: requires more than automatic application of rules, i.e., needs human ingenuity to complete
- ▶ Example: $a^2 + b^2 = c^2$

Computation vs. Deduction

- ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)
- ▶ Deduction: requires more than automatic application of rules, i.e., needs human ingenuity to complete
- ▶ Example: $a^2 + b^2 = c^2$... Pythagoras $\rightarrow$ first proof
- ▶ Another example: $a^n + b^n \neq c^n, n > 2$ (Fermat's last thm)

Computation vs. Deduction

- ▶ Computation: $10 + 0 = 10$ (using some predefined rules, i.e., $n + 0 = n$)
- ▶ Deduction: requires more than automatic application of rules, i.e., needs human ingenuity to complete
- ▶ Example: $a^2 + b^2 = c^2$... Pythagoras $\rightarrow$ first proof
- ▶ Another example: $a^n + b^n \neq c^n$, $n > 2$ (Fermat's last thm)
It took 358 years until it was proved!

Computation vs. Deduction

Computation vs. Deduction

- ▶ Is there a way to unify computation and deduction?

Computation vs. Deduction

- ▶ Is there a way to unify computation and deduction?
- ▶ G. Boole reduced a certain class of logical reasoning to computation in Boolean algebras

Computation vs. Deduction

- ▶ Is there a way to unify computation and deduction?
- ▶ G. Boole reduced a certain class of logical reasoning to computation in Boolean algebras
- ▶ However, the undecidability results in 20th century implies that certain facts are not mechanically computable

Computation vs. Deduction

- ▶ Is there a way to unify computation and deduction?
- ▶ G. Boole reduced a certain class of logical reasoning to computation in Boolean algebras
- ▶ However, the undecidability results in 20th century implies that certain facts are not mechanically computable
- ▶ Computation can be regarded as a limited form of deduction
 - ▶ $1 + 1 = 2$ is the result of a computation and a theorem of arithmetic

Computation vs. Deduction

- ▶ Is there a way to unify computation and deduction?
- ▶ G. Boole reduced a certain class of logical reasoning to computation in Boolean algebras
- ▶ However, the undecidability results in 20th century implies that certain facts are not mechanically computable
- ▶ Computation can be regarded as a limited form of deduction
 - ▶ $1 + 1 = 2$ is the result of a computation and a theorem of arithmetic
- ▶ Deduction can be seen as a form of computation if we fix a strategy for proof search, remove the guesswork...

Computation vs. Deduction

- ▶ Is there a way to unify computation and deduction?
- ▶ G. Boole reduced a certain class of logical reasoning to computation in Boolean algebras
- ▶ However, the undecidability results in 20th century implies that certain facts are not mechanically computable
- ▶ Computation can be regarded as a limited form of deduction
 - ▶ $1 + 1 = 2$ is the result of a computation and a theorem of arithmetic
- ▶ Deduction can be seen as a form of computation if we fix a strategy for proof search, remove the guesswork...
- ▶ The latter idea is the foundation of logic programming

Some background: FOL

Some background: FOL

Reminder:

Some background: FOL

Reminder:

- ▶ Terms:

- ▶ $f(t_1, \dots, t_n)$ – where f has arity n , and can contain variables

Some background: FOL

Reminder:

- ▶ Terms:
 - ▶ $f(t_1, \dots, t_n)$ – where f has arity n , and can contain variables
 - ▶ variables

Some background: FOL

Reminder:

- ▶ Terms:
 - ▶ $f(t_1, \dots, t_n)$ – where f has arity n , and can contain variables
 - ▶ variables
- ▶ Predicates: $p(t_1, \dots, t_n)$ – where p is a predicate of arity n

Some background: FOL

Reminder:

- ▶ Terms:
 - ▶ $f(t_1, \dots, t_n)$ – where f has arity n , and can contain variables
 - ▶ variables
- ▶ Predicates: $p(t_1, \dots, t_n)$ – where p is a predicate of arity n
- ▶ Formulas: $\phi := p(t_1, \dots, t_n) \mid \neg\phi \mid \phi \wedge \phi \mid \exists x.\phi$

Some background

Some background

Reminder:

Some background

Reminder:

- Inference rules:

$$\frac{C_1 \ C_2 \ \dots \ C_n}{H} \text{ RULENAME}$$

Some background

Reminder:

- Inference rules:

$$\frac{C_1 \ C_2 \ \dots \ C_n}{H} \text{RULENAME}$$

- Examples:

$$\frac{}{\text{nat}(0)} \text{ZERO}$$

$$\frac{\text{nat}(N)}{\text{nat}(s(N))} \text{SUCC}$$

Some background

Reminder:

- Inference rules:

$$\frac{C_1 \ C_2 \ \dots \ C_n}{H} \text{ RULENAME}$$

- Examples:

$$\frac{}{\text{nat}(0)} \text{ ZERO}$$

$$\frac{\text{nat}(N)}{\text{nat}(s(N))} \text{ SUCC}$$

$$\frac{}{\text{even}(0)} \text{ EVEN0}$$

$$\frac{\text{even}(N)}{\text{even}(s(s(N)))} \text{ EVENSUCC}$$

Proofs

- ▶ Proof that $\text{even}(s(s(s(s(0)))))$ holds:

$$\frac{\frac{\frac{\frac{\cdot}{\text{even}(0)} \text{EVEN0}}{\text{even}(s(s(0)))} \text{EVENSUCC}}{\text{even}(s(s(s(s(0))))} \text{EVENSUCC}}$$

Proofs

- ▶ Proof that $even(s(s(s(s(0))))$ holds:

$$\frac{\frac{\frac{\frac{\cdot}{even(0)} \text{ EVEN0}}{even(s(s(0)))} \text{ EVENSUCC}}{even(s(s(s(s(0))))} \text{ EVENSUCC}}$$

- ▶ Proof strategies: *goal-directed* vs. *forward-reasoning*

Proofs

- ▶ Proof that $even(s(s(s(s(0))))$ holds:

$$\frac{\frac{\frac{\frac{\cdot}{even(0)} \text{ EVEN0}}{even(s(s(0)))} \text{ EVENSUCC}}{even(s(s(s(s(0))))} \text{ EVENSUCC}}$$

- ▶ Proof strategies: *goal-directed* vs. *forward-reasoning*
- ▶ Logic programming uses *goal-directed*: for the current goal, search the rule that might have been applied to arrive at this conclusion

$$\frac{\cdot}{even(0)} \text{ EVEN0} \qquad \frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

Logic programming: general facts

- ▶ This paradigm is based on formal logic

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$
 - ▶ H is the *head* of the clause

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$
 - ▶ H is the *head* of the clause
 - ▶ $C_1, \dots, C_2$ is the *body*

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$
 - ▶ H is the *head* of the clause
 - ▶ $C_1, \dots, C_2$ is the *body*
 - ▶ $H.$ denotes a clause without body, a.k.a. *facts*

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$
 - ▶ H is the *head* of the clause
 - ▶ $C_1, \dots, C_2$ is the *body*
 - ▶ $H.$ denotes a clause without body, a.k.a. *facts*
- ▶ Logic programming can be seen as “controlled” deduction

Logic programming: general facts

- ▶ This paradigm is based on formal logic
- ▶ Programs are sets of sentences in the logical form of clauses (referred to as rules):
 - ▶ $H :- C_1, \dots, C_2.$
 - ▶ H is the *head* of the clause
 - ▶ $C_1, \dots, C_2$ is the *body*
 - ▶ $H.$ denotes a clause without body, a.k.a. *facts*
- ▶ Logic programming can be seen as “controlled” deduction
- ▶ Logic programming languages (or families of PL): Prolog, ASP (Answer set programming), Datalog

Logic programming: how it works

- ▶ Prolog: early 1970, Alain Colmerauer and Robert Kowalski
- ▶ Demo in Prolog: goals.

$$\frac{}{nat(0)} \text{ ZERO} \qquad \frac{nat(N)}{nat(s(N))} \text{ SUCC}$$

Logic programming: how it works

- ▶ Prolog: early 1970, Alain Colmerauer and Robert Kowalski
- ▶ Demo in Prolog: goals.

$$\frac{}{nat(0)} \text{ ZERO} \qquad \frac{nat(N)}{nat(s(N))} \text{ SUCC}$$

$$\frac{}{even(0)} \text{ EVEN0} \qquad \frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

- ▶ Proof search: goal-directed

Substitutions

- ▶ $\sigma = \{x \mapsto 3, y \mapsto y', z \mapsto f(a, b)\}$ - substitution

Substitutions

- ▶ $\sigma = \{x \mapsto 3, y \mapsto y', z \mapsto f(a, b)\}$ - substitution
- ▶ Let $T_1 = g(h(z, y), x, y)$ be a term

Substitutions

- ▶ $\sigma = \{x \mapsto 3, y \mapsto y', z \mapsto f(a, b)\}$ - substitution
- ▶ Let $T_1 = g(h(z, y), x, y)$ be a term
- ▶ Then, $\sigma(T_1) = \sigma(g(h(z, y), x, y)) = g(h(f(a, b), y'), 3, y')$.

Unification

- ▶ When proving, the conclusion of an inference rule matches the current goal.

Unification

- ▶ When proving, the conclusion of an inference rule matches the current goal.
- ▶ How?
- ▶ Unification
 - ▶ A substitution σ is the *unifier* of t_1 and t_2 if $\sigma(t_1) = \sigma(t_2)$.
- ▶ Various algorithms for unification (Martelli&Montanari, Robinson, ...)

Answer substitutions

- ▶ Unification $\rightarrow$ variables in goals, i.e, $even(s(s(X)))$.

Answer substitutions

- ▶ Unification $\rightarrow$ variables in goals, i.e, $even(s(s(X)))$.
- ▶ Demo in Prolog: queries.

$$\frac{}{even(0)} \text{ EVEN0}$$

$$\frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

Answer substitutions

- ▶ Unification $\rightarrow$ variables in goals, i.e, $even(s(s(X)))$.
- ▶ Demo in Prolog: queries.

$$\frac{}{even(0)} \text{ EVEN0}$$

$$\frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

Other aspects:

Answer substitutions

- ▶ Unification $\rightarrow$ variables in goals, i.e, $even(s(s(X)))$.
- ▶ Demo in Prolog: queries.

$$\frac{}{even(0)} \text{ EVEN0} \qquad \frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

Other aspects:

- ▶ Backtracking: more than one possible rule applicable

Answer substitutions

- ▶ Unification $\rightarrow$ variables in goals, i.e, $even(s(s(X)))$.
- ▶ Demo in Prolog: queries.

$$\frac{}{even(0)} \text{ EVEN0} \qquad \frac{even(N)}{even(s(s(N)))} \text{ EVENSUCC}$$

Other aspects:

- ▶ Backtracking: more than one possible rule applicable
- ▶ Subgoal order: which of the premises to consider?

Demo

Light meal problem

► Facts:

```
meat (steak, 10) .  
meat (pork, 12) .  
fish (tuna, 2) .  
fish (trout, 4) .  
starter (salad, 1) .  
starter (soup, 3) .  
desert (fruit, 1) .  
desert (cake, 10) .  
desert (icecream, 4) .
```

Demo

Light meal problem

► Main course:

```
main_course(N, K) :- meat(N, K); fish(N, K).
```


Demo

Light meal problem

- ▶ Main course:

```
main_course(N, K) :- meat(N, K); fish(N, K).
```

- ▶ A simple meal:

```
meal(S, M, D) :- starter(S, SK),  
                  main_course(M, MK),  
                  desert(D, DK).
```

Demo

Light meal problem

- ▶ Main course:

```
main_course(N, K) :- meat(N, K); fish(N, K).
```

- ▶ A simple meal:

```
meal(S, M, D) :- starter(S, SK),  
                  main_course(M, MK),  
                  desert(D, DK).
```

- ▶ A light meal:

```
light_meal(S, M, D) :- starter(S, SK),  
                        main_course(M, MK),  
                        desert(D, DK),  
                        SK + MK + DK < 10.
```

Resources

Reading material:

- ▶ **Chapter/Lecture 1:** <https://people.mpi-sws.org/~dg/teaching/lis2014/modules/lp-fp-07.pdf>